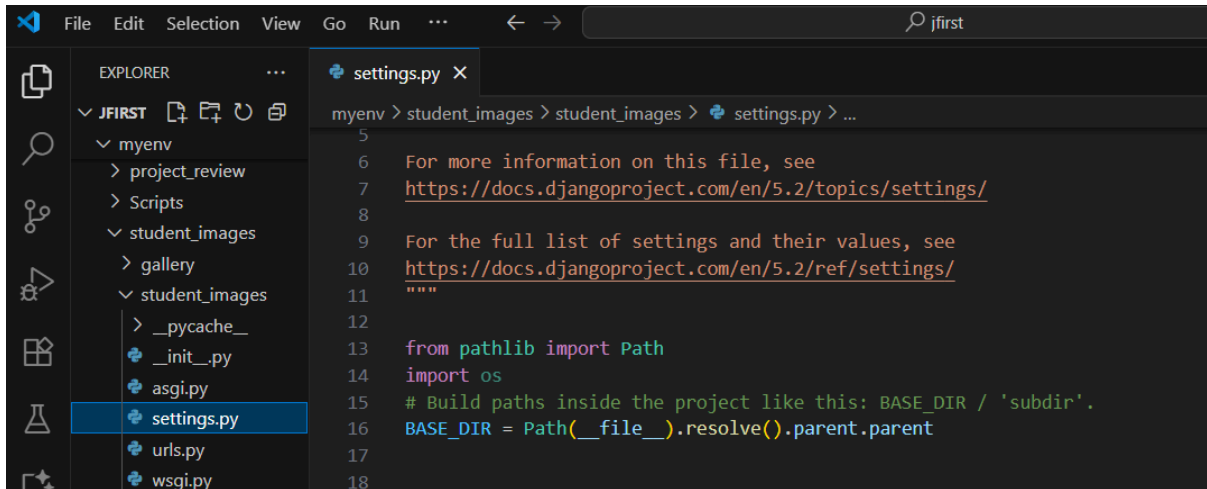
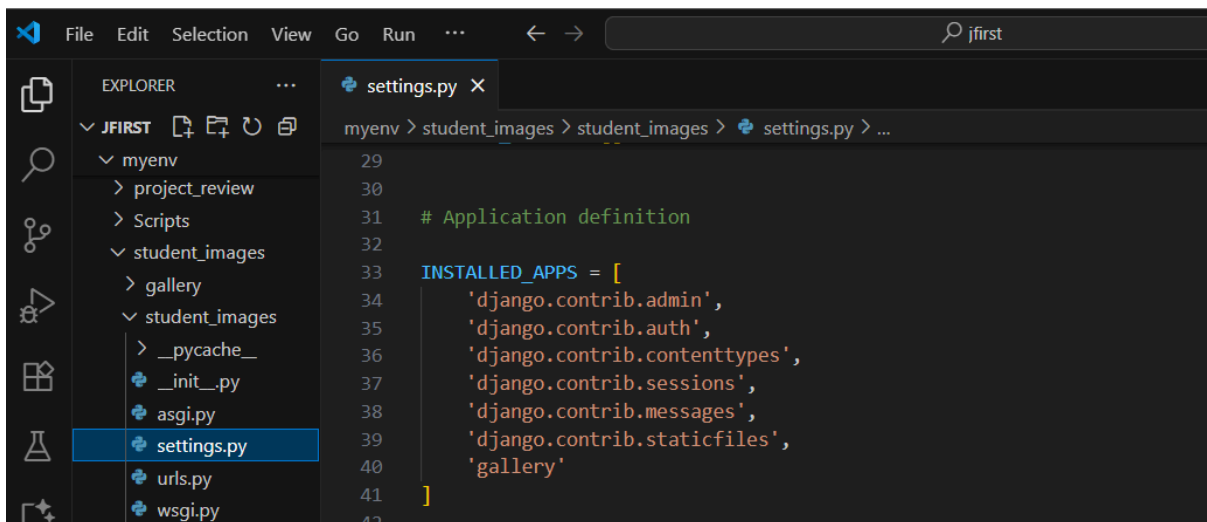


SETTINGS : student_images\settings.py



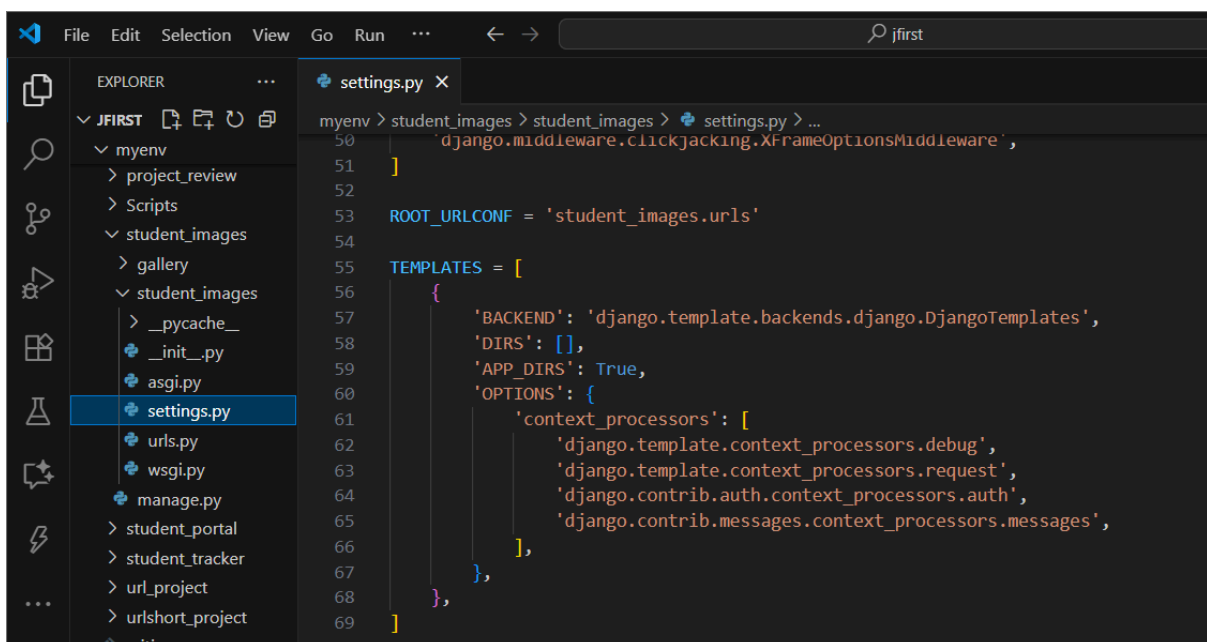
This screenshot shows the beginning of the `settings.py` file in a Visual Studio Code editor. The Explorer panel on the left shows the project structure with `student_images\settings.py` selected. The main editor window displays the following code:

```
5
6  For more information on this file, see
7  https://docs.djangoproject.com/en/5.2/topics/settings/
8
9  For the full list of settings and their values, see
10 https://docs.djangoproject.com/en/5.2/ref/settings/
11 """
12
13 from pathlib import Path
14 import os
15 # Build paths inside the project like this: BASE_DIR / 'subdir'.
16 BASE_DIR = Path(__file__).resolve().parent.parent
17
18
```



This screenshot shows the application definition section of the `settings.py` file. The Explorer panel on the left shows the project structure with `student_images\settings.py` selected. The main editor window displays the following code:

```
29
30
31 # Application definition
32
33 INSTALLED_APPS = [
34     'django.contrib.admin',
35     'django.contrib.auth',
36     'django.contrib.contenttypes',
37     'django.contrib.sessions',
38     'django.contrib.messages',
39     'django.contrib.staticfiles',
40     'gallery'
41 ]
42
```



This screenshot shows the end of the `settings.py` file in a Visual Studio Code editor. The Explorer panel on the left shows the project structure with `student_images\settings.py` selected. The main editor window displays the following code:

```
50     'django.middleware.clickjacking.XFrameOptionsMiddleware',
51 ]
52
53 ROOT_URLCONF = 'student_images.urls'
54
55 TEMPLATES = [
56     {
57         'BACKEND': 'django.template.backends.django.DjangoTemplates',
58         'DIRS': [],
59         'APP_DIRS': True,
60         'OPTIONS': {
61             'context_processors': [
62                 'django.template.context_processors.debug',
63                 'django.template.context_processors.request',
64                 'django.contrib.auth.context_processors.auth',
65                 'django.contrib.messages.context_processors.messages',
66             ],
67         },
68     },
69 ]
70
```

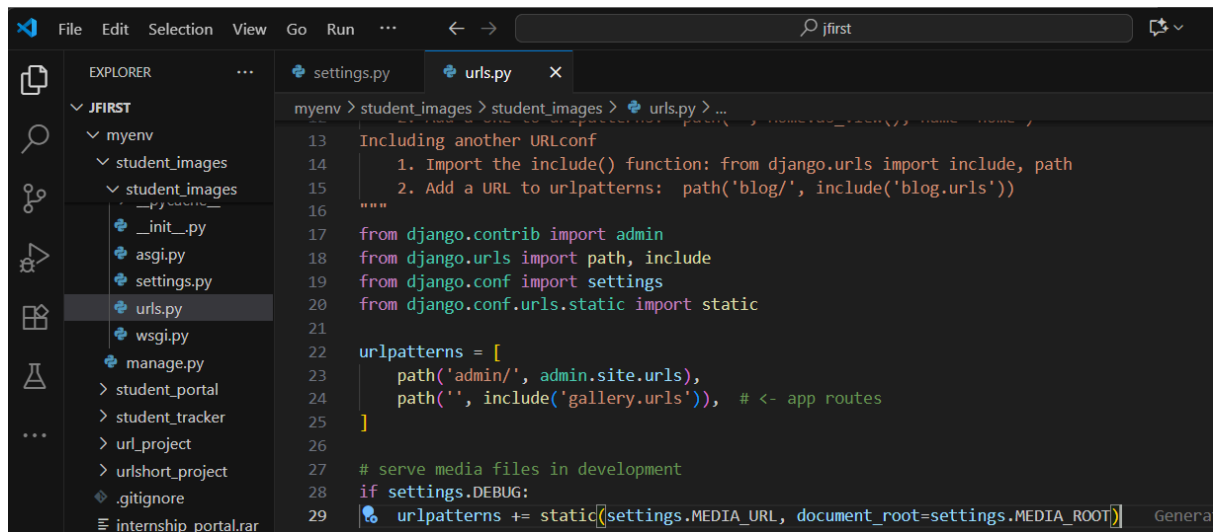
```
116
117 # Static files (CSS, JavaScript, Images)
118 # https://docs.djangoproject.com/en/5.2/howto/static-files/
119
120 STATIC_URL = 'static/'
121 STATIC_ROOT = BASE_DIR / 'static'
122 STATICFILES_DIRS = []
123
124 MEDIA_URL = '/media/'
125 MEDIA_ROOT = BASE_DIR / 'media'
126 # Default primary key field type
127 # https://docs.djangoproject.com/en/5.2/ref/settings/#default-auto-field
128
129 DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
130
```

Generate code: Alt+Shift+Enter

MODELS : gallery\models.py

```
1 from django.db import models
2
3 class StudentPhoto(models.Model):
4     name = models.CharField(max_length=120)
5     photo = models.ImageField(upload_to='originals/')
6     uploaded_at = models.DateTimeField(auto_now_add=True)
7
8     def __str__(self):
9         return f"{self.name} ({self.id})"
10
11 class ProcessedImage(models.Model):
12     OP_CHOICES = [ ('resize', 'Resize'), ('grayscale', 'Grayscale'),
13                   ('resize+grayscale', 'Resize + Grayscale'), ('original-copy', 'Original Copy'),
14                   ]
15     student_photo = models.ForeignKey(
16         StudentPhoto, on_delete=models.CASCADE, related_name='processed' )
17     image = models.ImageField(upload_to='processed/')
18     operation = models.CharField(max_length=50, choices=OP_CHOICES)
19     width = models.PositiveIntegerField(null=True, blank=True)
20     height = models.PositiveIntegerField(null=True, blank=True)
21     created_at = models.DateTimeField(auto_now_add=True)
22
23     class Meta:
24         ordering = ['-created_at']
25
26     def __str__(self):
27         return f"{self.student_photo.name} - {self.operation} ({self.id})"
```

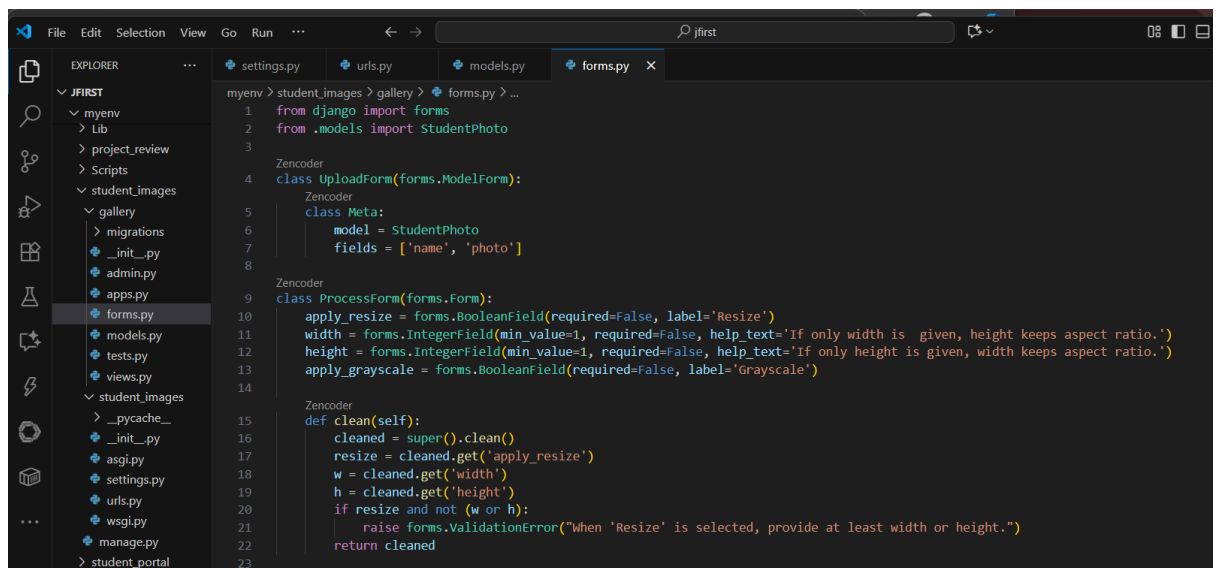
URLS : student_images\urls.py



The screenshot shows the Visual Studio Code editor with the Explorer sidebar on the left. The Explorer shows a project structure with a folder named 'myenv' containing a subfolder 'student_images'. Inside 'student_images', there are files: '_init_.py', 'asgi.py', 'settings.py', 'urls.py' (selected), and 'wsgi.py'. The main editor window displays the content of 'urls.py'.

```
13 Including another URLconf
14 1. Import the include() function: from django.urls import include, path
15 2. Add a URL to urlpatterns: path('blog/', include('blog.urls'))
16 """
17 from django.contrib import admin
18 from django.urls import path, include
19 from django.conf import settings
20 from django.conf.urls.static import static
21
22 urlpatterns = [
23     path('admin/', admin.site.urls),
24     path('', include('gallery.urls')), # <- app routes
25 ]
26
27 # serve media files in development
28 if settings.DEBUG:
29     urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

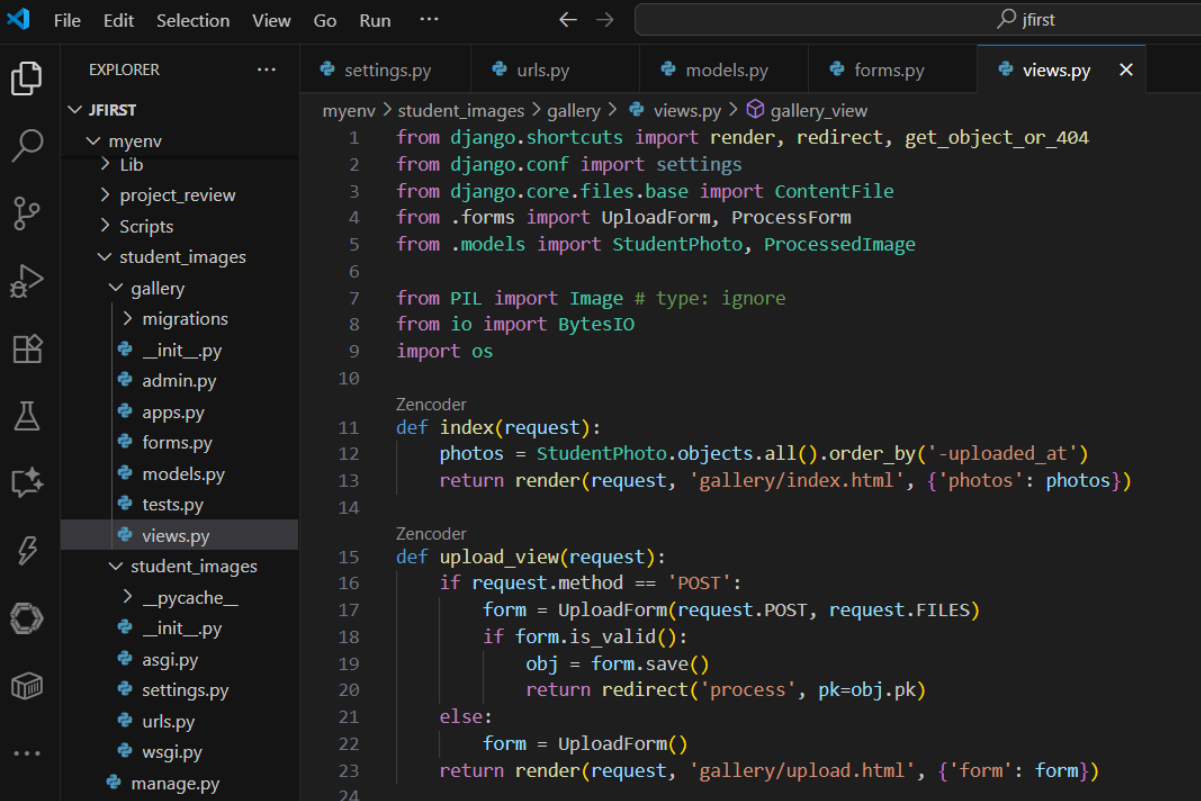
FORMS : gallery\forms.py



The screenshot shows the Visual Studio Code editor with the Explorer sidebar on the left. The Explorer shows a project structure with a folder named 'myenv' containing a subfolder 'student_images'. Inside 'student_images', there is a subfolder 'gallery' which contains files: 'migrations', '_init_.py', 'admin.py', 'apps.py', 'forms.py' (selected), 'models.py', 'tests.py', 'views.py', and 'wsgi.py'. The main editor window displays the content of 'forms.py'.

```
1 from django import forms
2 from .models import StudentPhoto
3
4 class UploadForm(forms.ModelForm):
5     class Meta:
6         model = StudentPhoto
7         fields = ['name', 'photo']
8
9 class ProcessForm(forms.Form):
10     apply_resize = forms.BooleanField(required=False, label='Resize')
11     width = forms.IntegerField(min_value=1, required=False, help_text='If only width is given, height keeps aspect ratio.')
12     height = forms.IntegerField(min_value=1, required=False, help_text='If only height is given, width keeps aspect ratio.')
13     apply_grayscale = forms.BooleanField(required=False, label='Grayscale')
14
15     def clean(self):
16         cleaned = super().clean()
17         resize = cleaned.get('apply_resize')
18         w = cleaned.get('width')
19         h = cleaned.get('height')
20         if resize and not (w or h):
21             raise forms.ValidationError("When 'Resize' is selected, provide at least width or height.")
22         return cleaned
```

VIEWS : gallery\views.py



```
File Edit Selection View Go Run ... jfirst

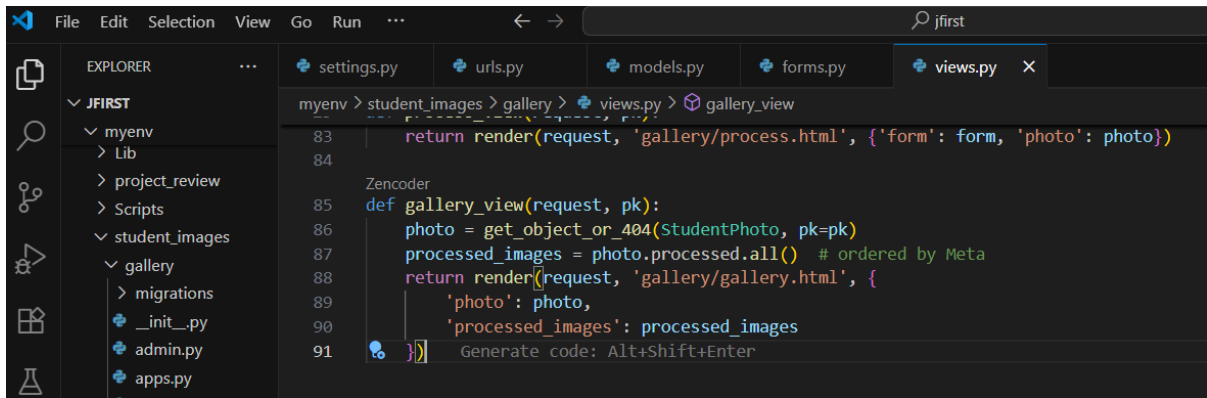
EXPLORER
  JFIRST
    myenv
      Lib
      project_review
      Scripts
    student_images
      gallery
        migrations
        _init_.py
        admin.py
        apps.py
        forms.py
        models.py
        tests.py
        views.py
      student_images
        __pycache__
        _init_.py
        asgi.py
        settings.py
        urls.py
        wsgi.py
        manage.py

settings.py urls.py models.py forms.py views.py X

myenv > student_images > gallery > views.py > gallery_view
1  from django.shortcuts import render, redirect, get_object_or_404
2  from django.conf import settings
3  from django.core.files.base import ContentFile
4  from .forms import UploadForm, ProcessForm
5  from .models import StudentPhoto, ProcessedImage
6
7  from PIL import Image # type: ignore
8  from io import BytesIO
9  import os
10
11  Zencoder
12  def index(request):
13      photos = StudentPhoto.objects.all().order_by('-uploaded_at')
14      return render(request, 'gallery/index.html', {'photos': photos})
15
16  Zencoder
17  def upload_view(request):
18      if request.method == 'POST':
19          form = UploadForm(request.POST, request.FILES)
20          if form.is_valid():
21              obj = form.save()
22              return redirect('process', pk=obj.pk)
23      else:
24          form = UploadForm()
25      return render(request, 'gallery/upload.html', {'form': form})
```

```
File Edit Selection View Go Run ... jfirst
EXPLORER
JFIRST
  myenv
  Lib
  project_review
  Scripts
  student_images
    gallery
      migrations
      _init_.py
      admin.py
      apps.py
      forms.py
      models.py
      tests.py
      views.py
    student_images
      __pycache__
      _init_.py
      asgi.py
      settings.py
      urls.py
      wsgi.py
      manage.py
      student_portal
      student_tracker
      url_project
      urlshort_project
      .gitignore
  settings.py
  urls.py
  models.py
  forms.py
  views.py
myenv > student_images > gallery > views.py > gallery_view
23 return render(request, 'gallery/upload.html', {'form': form})
24
25 def process_view(request, pk):
26     photo = get_object_or_404(StudentPhoto, pk=pk)
27     if request.method == 'POST':
28         form = ProcessForm(request.POST)
29         if form.is_valid():
30             img = Image.open(photo.photo.path)
31
32             # Normalize base image mode
33             # (keep "L" if already grayscale after conversion below)
34             img = img.convert('RGB')
35
36             ops = []
37             w_val = None
38             h_val = None
39             if form.cleaned_data['apply_resize']:
40                 w = form.cleaned_data['width']
41                 h = form.cleaned_data['height']
42
43                 # aspect-ratio preserving calculation if one is missing
44                 if w and not h:
45                     ratio = w / img.width
46                     h = int(img.height * ratio)
47                 elif h and not w:
48                     ratio = h / img.height
49                     w = int(img.width * ratio)
50                 elif not (w and h):
51                     w, h = img.size
52
```

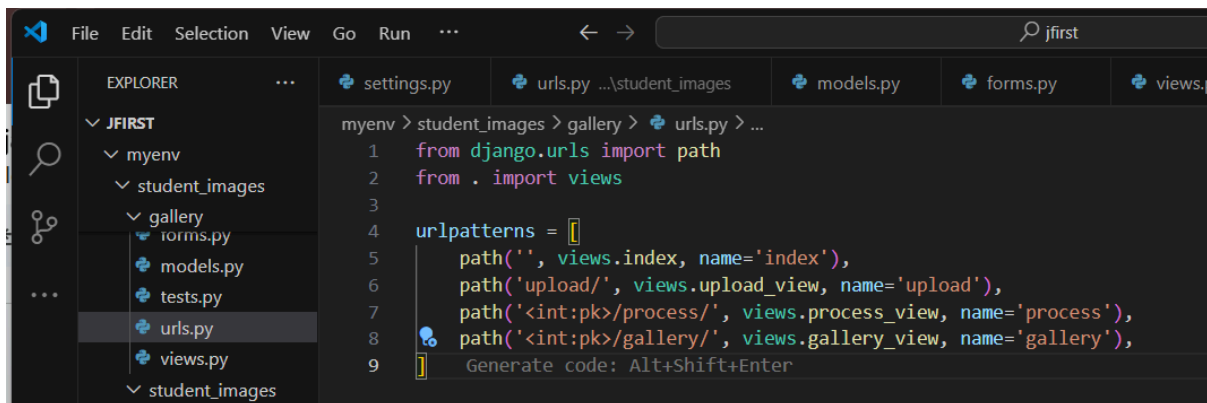
```
25 def process_view(request, pk):
52
53     img = img.resize(w, h)
54     ops.append('resize')
55     w_val, h_val = w, h
56
57     if form.cleaned_data['apply_grayscale']:
58         img = img.convert('L') # grayscale
59         ops.append('grayscale')
60
61     operation = '+'.join(ops) if ops else 'original-copy'
62
63     # Save to memory (JPEG works fine for RGB and L)
64     buffer = BytesIO()
65     img.save(buffer, format='JPEG', quality=90)
66     buffer.seek(0)
67
68     base_name = os.path.splitext(os.path.basename(photo.photo.name))[0]
69     file_name = f"{base_name}_{operation}.jpg"
70
71     processed = ProcessedImage(
72         student_photo=photo,
73         operation=operation,
74         width=w_val,
75         height=h_val
76     )
77     processed.image.save(file_name, ContentFile(buffer.getvalue()), save=True)
78
79     return redirect('gallery', pk=photo.pk)
80 else:
81     form = ProcessForm()
82
83     return render(request, 'gallery/process.html', {'form': form, 'photo': photo})
84
```



The screenshot shows the VS Code editor with the Explorer sidebar on the left. The Explorer sidebar shows a project structure with folders like 'myenv', 'Lib', 'project_review', 'Scripts', 'student_images', and 'gallery'. The 'gallery' folder is expanded, showing files like 'migrations', '_init_.py', 'admin.py', and 'apps.py'. The main editor area shows the 'views.py' file with the following code:

```
83 return render(request, 'gallery/process.html', {'form': form, 'photo': photo})
84
85 def gallery_view(request, pk):
86     photo = get_object_or_404(StudentPhoto, pk=pk)
87     processed_images = photo.processed.all() # ordered by Meta
88     return render(request, 'gallery/gallery.html', {
89         'photo': photo,
90         'processed_images': processed_images
91     })
```

URLS : gallery\urls.py

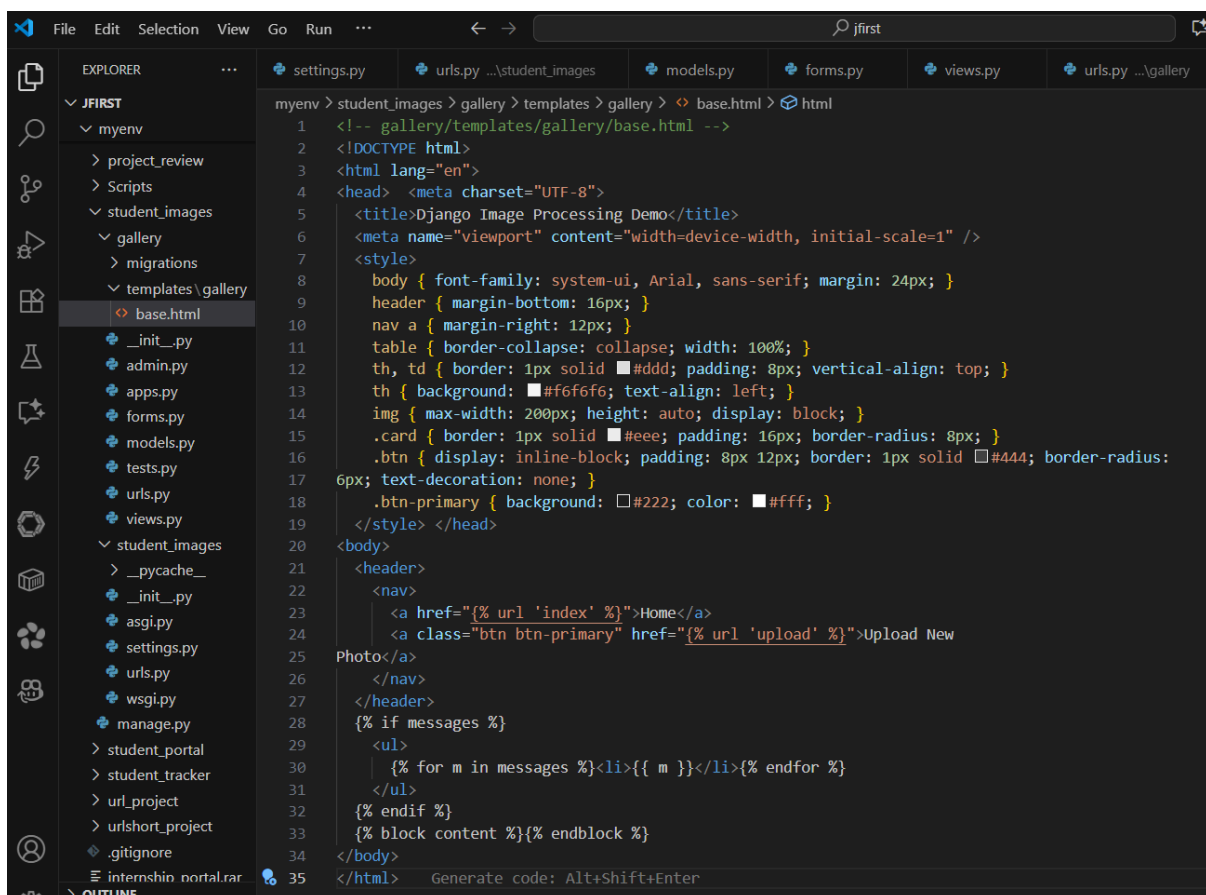


The screenshot shows the VS Code editor with the Explorer sidebar on the left. The Explorer sidebar shows a project structure with folders like 'myenv', 'student_images', 'gallery', 'forms.py', 'models.py', 'tests.py', 'urls.py', and 'views.py'. The 'urls.py' file is selected in the Explorer sidebar. The main editor area shows the 'urls.py' file with the following code:

```
1 from django.urls import path
2 from . import views
3
4 urlpatterns = [
5     path('', views.index, name='index'),
6     path('upload/', views.upload_view, name='upload'),
7     path('<int:pk>/process/', views.process_view, name='process'),
8     path('<int:pk>/gallery/', views.gallery_view, name='gallery'),
9 ]
```

TEMPLATES :

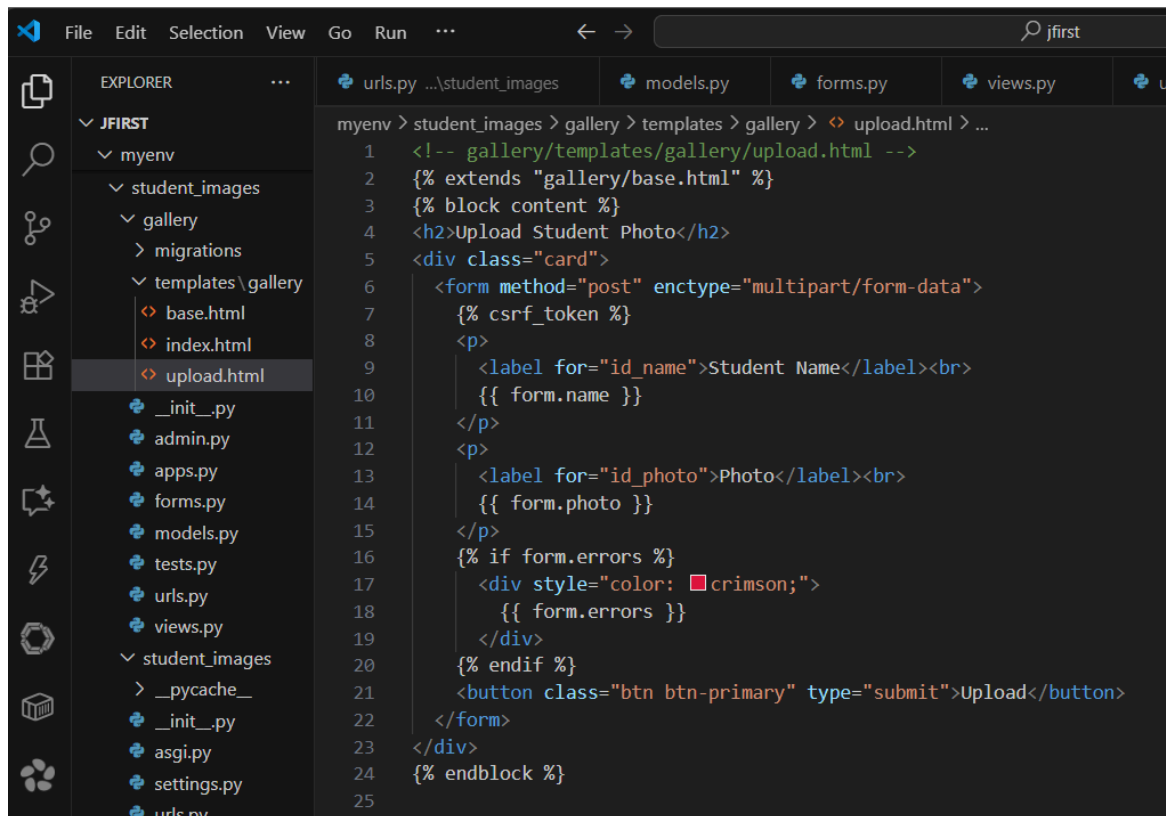
templates\gallery\base.html



The screenshot shows the VS Code editor with the Explorer sidebar on the left. The Explorer sidebar shows a project structure with folders like 'myenv', 'student_images', 'gallery', 'templates', 'gallery', 'base.html', '._init_.py', 'admin.py', 'apps.py', 'forms.py', 'models.py', 'tests.py', 'urls.py', 'views.py', 'student_images', '._pycache_', '._init_.py', 'asgi.py', 'settings.py', 'urls.py', 'wsgi.py', 'manage.py', 'student_portal', 'student_tracker', 'url_project', 'urlshort_project', '._gitignore', and 'internship_portal.rar'. The 'base.html' file is selected in the Explorer sidebar. The main editor area shows the 'base.html' file with the following code:

```
1 <!-- gallery/templates/gallery/base.html -->
2 <!DOCTYPE html>
3 <html lang="en">
4 <head> <meta charset="UTF-8">
5 <title>Django Image Processing Demo</title>
6 <meta name="viewport" content="width=device-width, initial-scale=1" />
7 <style>
8     body { font-family: system-ui, Arial, sans-serif; margin: 24px; }
9     header { margin-bottom: 16px; }
10    nav a { margin-right: 12px; }
11    table { border-collapse: collapse; width: 100%; }
12    th, td { border: 1px solid #ddd; padding: 8px; vertical-align: top; }
13    th { background: #f6f6f6; text-align: left; }
14    img { max-width: 200px; height: auto; display: block; }
15    .card { border: 1px solid #eee; padding: 16px; border-radius: 8px; }
16    .btn { display: inline-block; padding: 8px 12px; border: 1px solid #444; border-radius:
17    6px; text-decoration: none; }
18    .btn-primary { background: #222; color: #fff; }
19 </style> </head>
20 <body>
21 <header>
22 <nav>
23 <a href="{% url 'index' %}">Home</a>
24 <a class="btn btn-primary" href="{% url 'upload' %}">Upload New
25 Photo</a>
26 </nav>
27 </header>
28 {% if messages %}
29 <ul>
30     {% for m in messages %}<li>{{ m }}</li>{% endfor %}
31 </ul>
32 {% endif %}
33 {% block content %}{% endblock %}
34 </body>
35 </html>
```

templates\gallery\upload.html



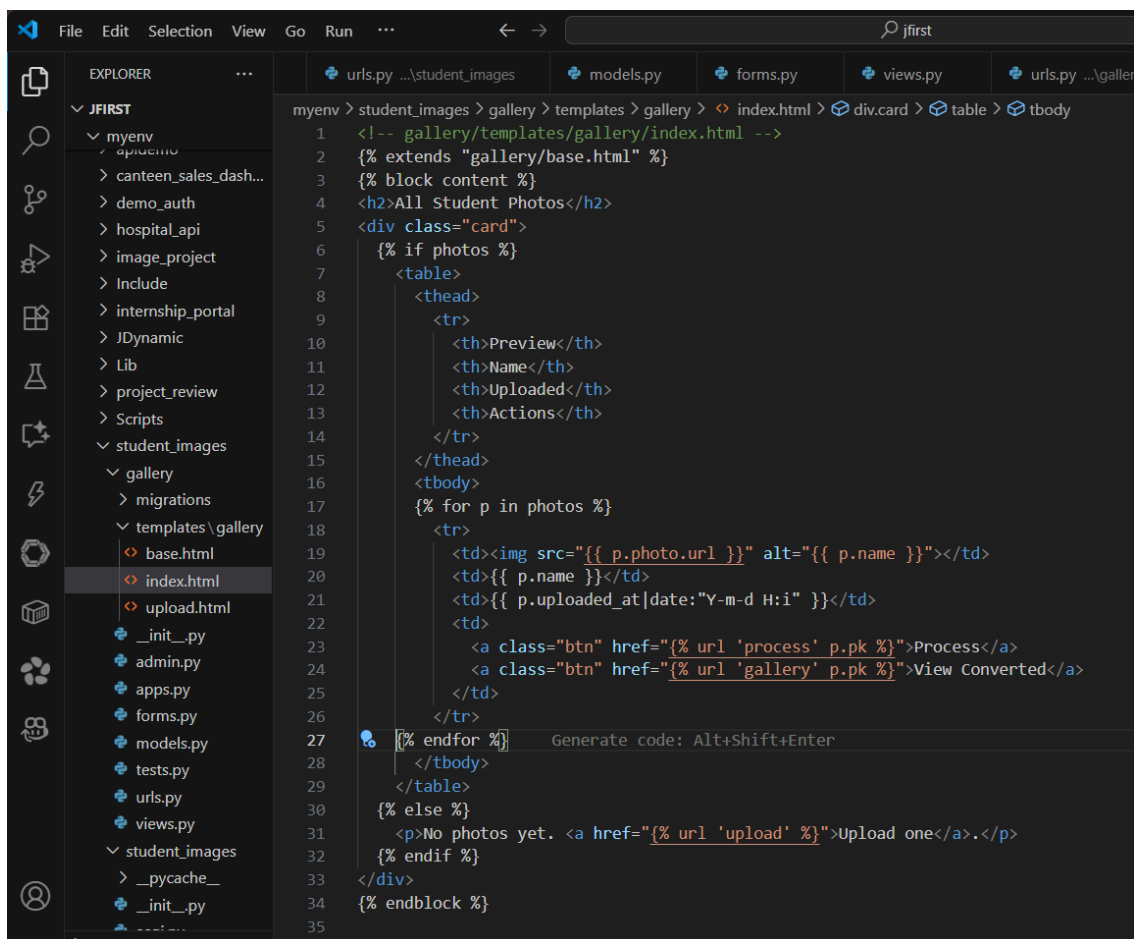
The screenshot shows the Visual Studio Code editor with the file explorer on the left and the code editor in the center. The file explorer shows the project structure with the following folders and files:

- JFIRST
 - myenv
 - student_images
 - gallery
 - migrations
 - templates\gallery
 - base.html
 - index.html
 - upload.html
- _init_.py
- admin.py
- apps.py
- forms.py
- models.py
- tests.py
- urls.py
- views.py
- student_images
 - __pycache__
 - _init_.py
 - asgi.py
 - settings.py
 - urls.py

The code editor shows the content of the `upload.html` file:

```
1 <!-- gallery/templates/gallery/upload.html -->
2 {% extends "gallery/base.html" %}
3 {% block content %}
4 <h2>Upload Student Photo</h2>
5 <div class="card">
6     <form method="post" enctype="multipart/form-data">
7         {% csrf_token %}
8         <p>
9             <label for="id_name">Student Name</label><br>
10            {{ form.name }}
11        </p>
12        <p>
13            <label for="id_photo">Photo</label><br>
14            {{ form.photo }}
15        </p>
16        {% if form.errors %}
17            <div style="color: crimson;">
18                {{ form.errors }}
19            </div>
20        {% endif %}
21        <button class="btn btn-primary" type="submit">Upload</button>
22    </form>
23 </div>
24 {% endblock %}
25
```

templates\gallery\index.html



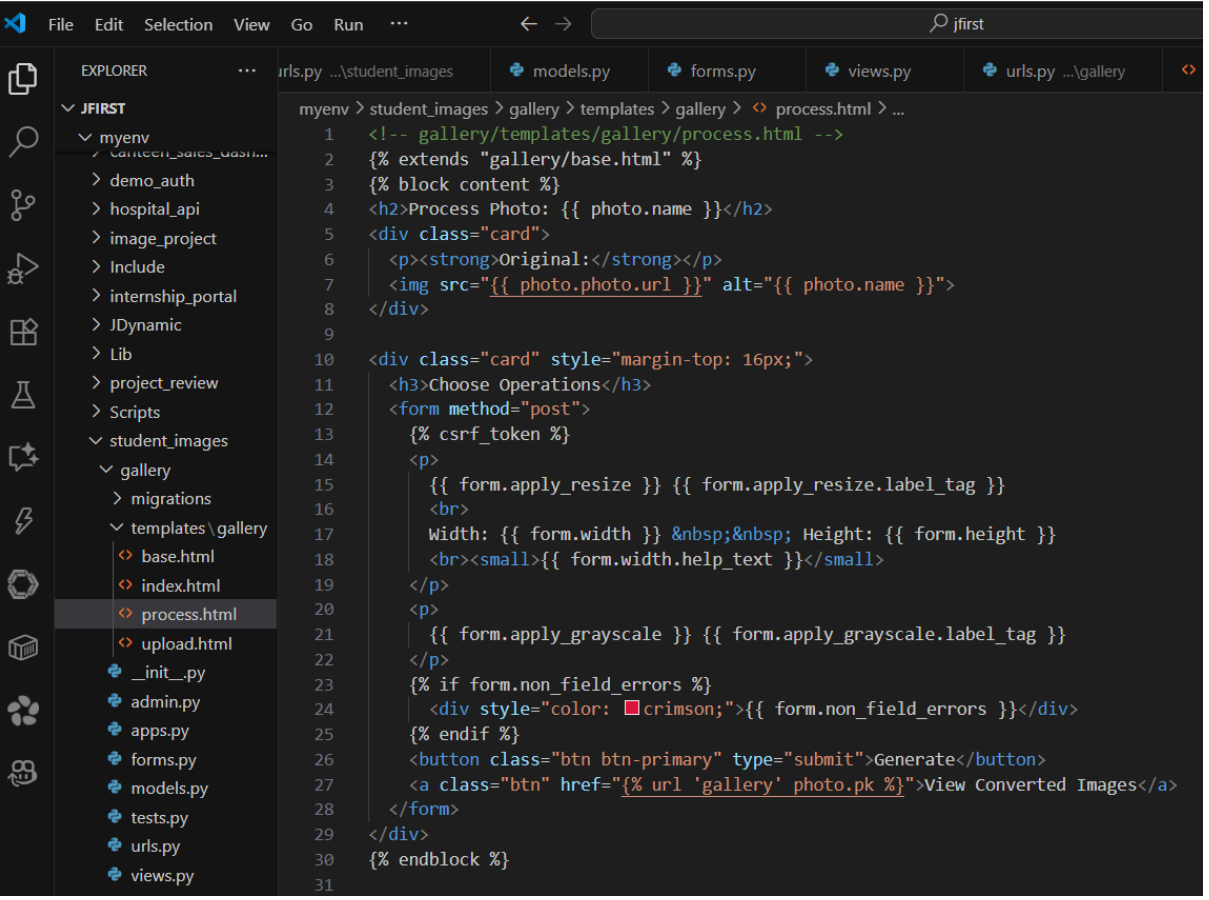
The screenshot shows the Visual Studio Code editor with the file explorer on the left and the code editor in the center. The file explorer shows the project structure with the following folders and files:

- JFIRST
 - myenv
 - student_images
 - gallery
 - migrations
 - templates\gallery
 - base.html
 - index.html
 - upload.html
 - _init_.py
 - admin.py
 - apps.py
 - forms.py
 - models.py
 - tests.py
 - urls.py
 - views.py
 - student_images
 - __pycache__
 - _init_.py
 - asgi.py
 - settings.py
 - urls.py

The code editor shows the content of the `index.html` file:

```
1 <!-- gallery/templates/gallery/index.html -->
2 {% extends "gallery/base.html" %}
3 {% block content %}
4 <h2>All Student Photos</h2>
5 <div class="card">
6     {% if photos %}
7         <table>
8             <thead>
9                 <tr>
10                    <th>Preview</th>
11                    <th>Name</th>
12                    <th>Uploaded</th>
13                    <th>Actions</th>
14                </tr>
15            </thead>
16            <tbody>
17                {% for p in photos %}
18                    <tr>
19                        <td></td>
20                        <td>{{ p.name }}</td>
21                        <td>{{ p.uploaded_at|date:"Y-m-d H:i" }}</td>
22                        <td>
23                            <a class="btn" href="{% url 'process' p.pk %}">Process</a>
24                            <a class="btn" href="{% url 'gallery' p.pk %}">View Converted</a>
25                        </td>
26                    </tr>
27                {% endfor %}
28            </tbody>
29        </table>
30        {% else %}
31            <p>No photos yet. <a href="{% url 'upload' %}">Upload one</a>.</p>
32        {% endif %}
33    </div>
34 {% endblock %}
35
```

templates\gallery\process.html



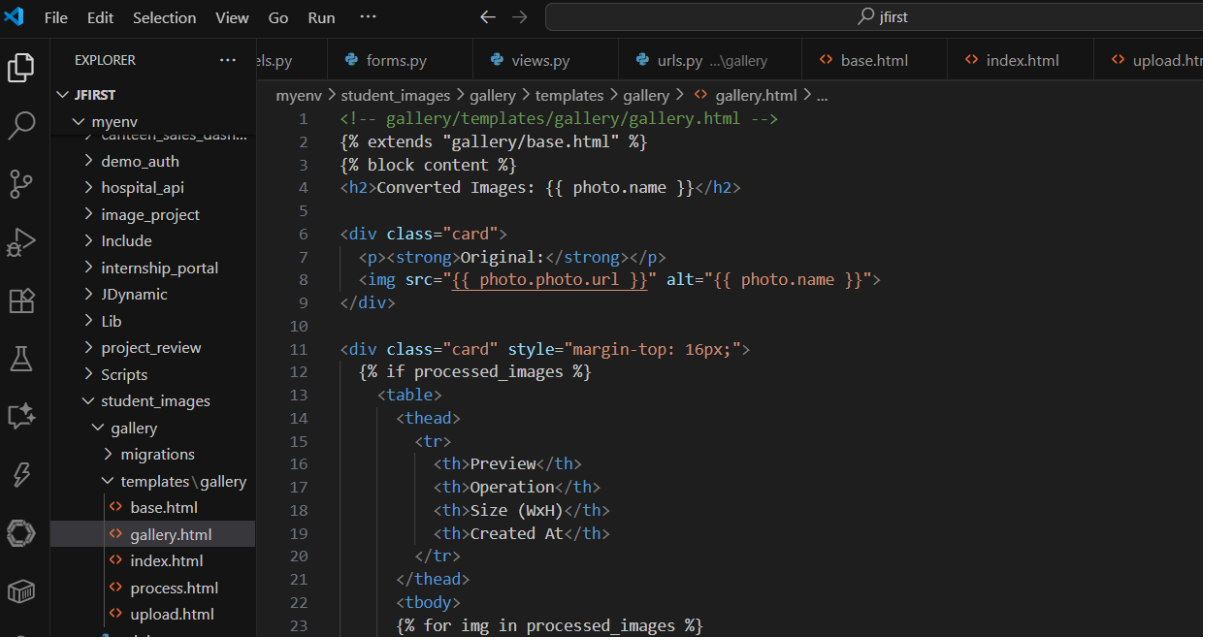
The screenshot shows the VS Code editor with the file explorer on the left and the code editor in the center. The file explorer shows the project structure with the following folders and files:

- myenv
 - demo_auth
 - hospital_api
 - image_project
 - Include
 - internship_portal
 - JDynamic
 - Lib
 - project_review
 - Scripts
- student_images
 - gallery
 - migrations
 - templates\gallery
 - base.html
 - index.html
 - process.html
 - upload.html
 - __init__.py
 - admin.py
 - apps.py
 - forms.py
 - models.py
 - tests.py
 - urls.py
 - views.py

The code editor shows the content of the process.html file:

```
1 <!-- gallery/templates/gallery/process.html -->
2 {% extends "gallery/base.html" %}
3 {% block content %}
4 <h2>Process Photo: {{ photo.name }}</h2>
5 <div class="card">
6   <p><strong>Original:</strong></p>
7   
8 </div>
9
10 <div class="card" style="margin-top: 16px;">
11   <h3>Choose Operations</h3>
12   <form method="post">
13     {% csrf_token %}
14     <p>
15       {{ form.apply_resize }} {{ form.apply_resize.label_tag }}
16       <br>
17       Width: {{ form.width }} &nbsp;&nbsp;&nbsp; Height: {{ form.height }}
18       <br><small>{{ form.width.help_text }}</small>
19     </p>
20     <p>
21       {{ form.apply_grayscale }} {{ form.apply_grayscale.label_tag }}
22     </p>
23     {% if form.non_field_errors %}
24       <div style="color: red;">{{ form.non_field_errors }}</div>
25     {% endif %}
26     <button class="btn btn-primary" type="submit">Generate</button>
27     <a class="btn" href="{% url 'gallery' photo.pk %}">View Converted Images</a>
28   </form>
29 </div>
30 {% endblock %}
31
```

templates\gallery\gallery.html



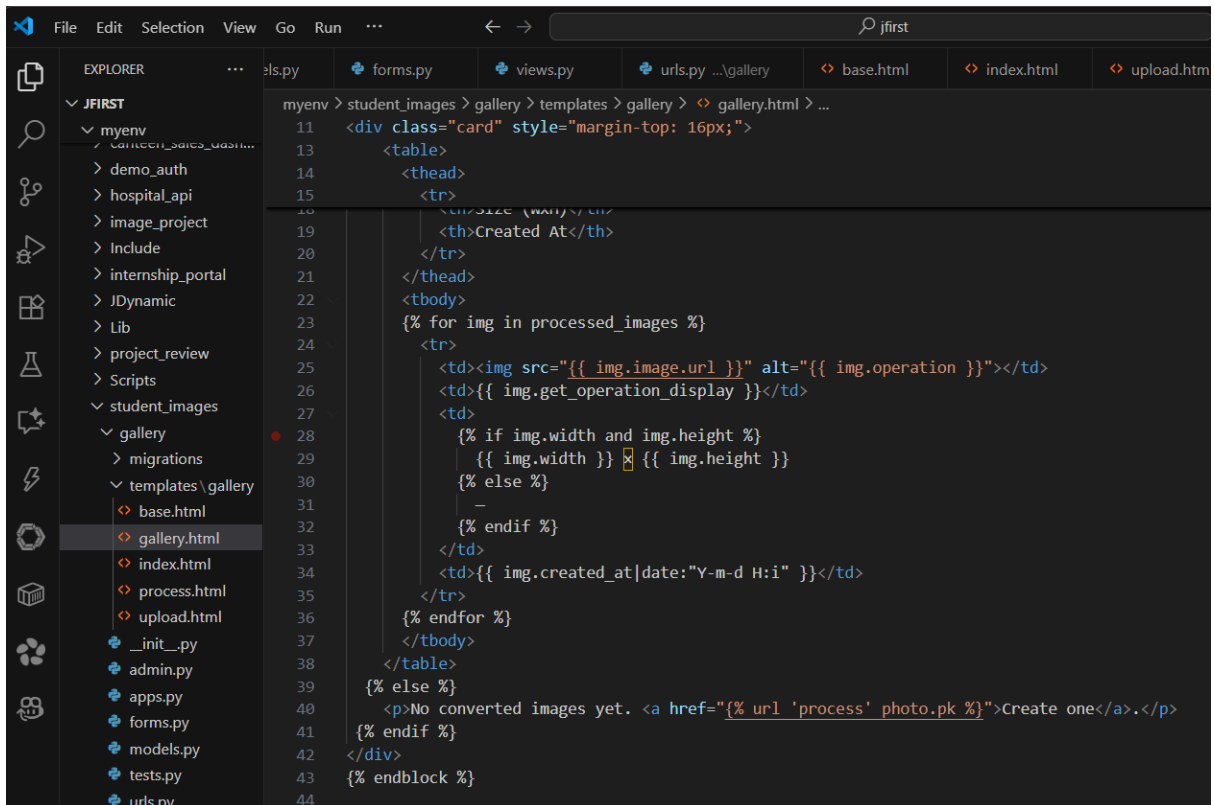
The screenshot shows the VS Code editor with the file explorer on the left and the code editor in the center. The file explorer shows the project structure with the following folders and files:

- myenv
 - demo_auth
 - hospital_api
 - image_project
 - Include
 - internship_portal
 - JDynamic
 - Lib
 - project_review
 - Scripts
- student_images
 - gallery
 - migrations
 - templates\gallery
 - base.html
 - gallery.html
 - index.html
 - process.html
 - upload.html
 - __init__.py
 - admin.py
 - apps.py
 - forms.py
 - models.py
 - tests.py
 - urls.py
 - views.py

The code editor shows the content of the gallery.html file:

```
1 <!-- gallery/templates/gallery/gallery.html -->
2 {% extends "gallery/base.html" %}
3 {% block content %}
4 <h2>Converted Images: {{ photo.name }}</h2>
5
6 <div class="card">
7   <p><strong>Original:</strong></p>
8   
9 </div>
10
11 <div class="card" style="margin-top: 16px;">
12   {% if processed_images %}
13     <table>
14       <thead>
15         <tr>
16           <th>Preview</th>
17           <th>Operation</th>
18           <th>Size (WxH)</th>
19           <th>Created At</th>
20         </tr>
21       </thead>
22       <tbody>
23         {% for img in processed_images %}

```



The screenshot shows a VS Code editor with a project named 'jfirst'. The Explorer sidebar on the left shows the project structure, including a 'student_images' app with a 'gallery' subdirectory. The main editor displays the 'gallery.html' template file, which contains Django template code for rendering a gallery of images. The code includes a loop over 'processed_images' and a conditional block for displaying the image or a message to upload one.

```
11 <div class="card" style="margin-top: 16px;">
12 <table>
13 <thead>
14 <tr>
15 <th>Created At</th>
16 </tr>
17 </thead>
18 <tbody>
19 {% for img in processed_images %}
20 <tr>
21 <td></td>
22 <td>{{ img.get_operation_display }}</td>
23 <td>
24 {% if img.width and img.height %}
25 {{ img.width }} x {{ img.height }}
26 {% else %}
27 -
28 {% endif %}
29 </td>
30 <td>{{ img.created_at|date:"Y-m-d H:i" }}</td>
31 </tr>
32 {% endfor %}
33 </tbody>
34 </table>
35 {% else %}
36 <p>No converted images yet. <a href="{% url 'process' photo.pk %}">Create one</a>.</p>
37 {% endif %}
38 </div>
39 {% endblock %}
```

OUTPUT SCREENS :

[Home](#)

[Upload New Photo](#)

All Student Photos

No photos yet. [Upload one.](#)

[Home](#)

[Upload New Photo](#)

Upload Student Photo

Student Name

Photo

Screenshot ...8 163751.png

Choose Operations

☒ Resize:

Width: Height:

If only width is given, height keeps aspect ratio.

☒ Grayscale:

[View Converted Images](#)

[Home](#) [Upload New Photo](#)

Converted Images: Divyadharshini

Original:			
			
Preview	Operation	Size (WxH)	Created At
	Resize + Grayscale	100 x 200	2025-09-28 11:12